

Advanced Machine Learning

DATA 642

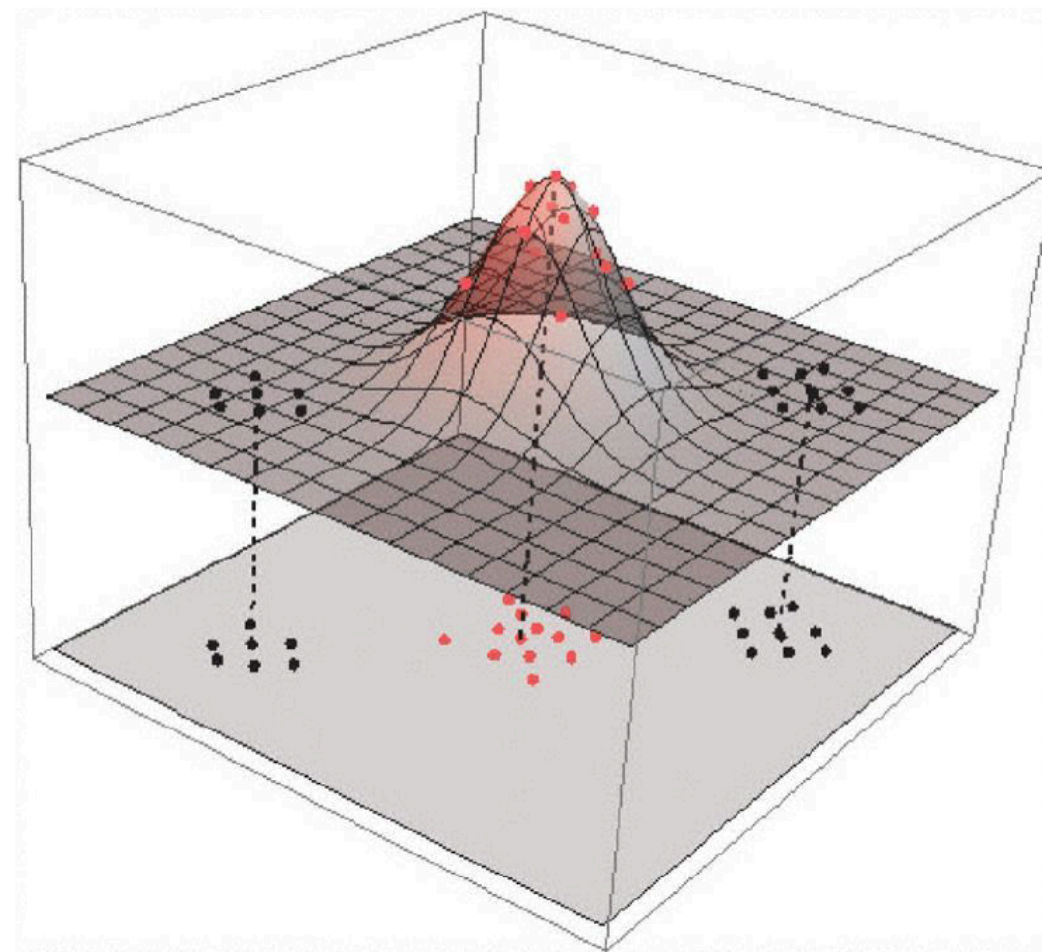
Lecture 5 — Reproducing Hilbert Spaces

*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Central idea

- In many practical applications, data points (observations) live in a non-linear space. One way to deal with **non-linear** modeling tasks is to map input variables to a new space, such that the originally non-linear task is transformed into a **linear-one**. Selecting this mapping though is not a straight-forward task.
- Given training set $(y_n, \mathbf{x}_n)$ for $n = 1, \dots, N$ we apply a function $\phi : \mathbb{R}^l \rightarrow \mathbb{R}^K$ with $K \gg l$ such that $\phi(\mathbf{x}_n)$ are linearly separable in $\mathbb{R}^K$.

- The points (red in one class and black for the other) that are not linearly separable in the original two-dimensional plane become linearly separable after the nonlinear mapping in the three-dimensional space; one can draw a plane that separates the "black" from the "red" points. In other words, after the mapping, we embed an l -dimensional manifold in a K -dimensional space in such a way that the data in the two classes become linearly separable.
- However, it is not easy to work in high-dimensional spaces. A large number of parameters are needed; this in turn poses computational complexity problems, and raises issues related to the generalization and over-fitting performance of the designed predictors.



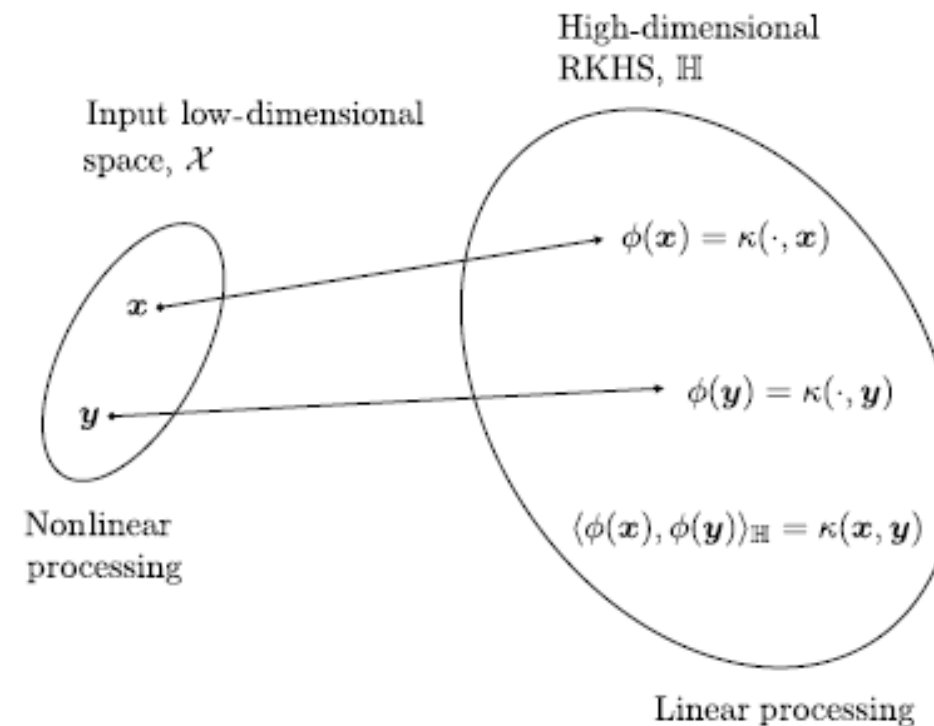
The process and some theoretical considerations

- Instead of explicitly defining a non-linear feature map $\phi(\cdot)$ and computing the resulting inner product between training samples $\mathbf{x}_i$ and $\mathbf{x}_j$, we define a similarity function $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ between $\mathbf{x}_i$ and $\mathbf{x}_j$. For a certain class of similarity functions, called **kernels**, the similarity function implicitly defines a non-linear feature map $\phi(\cdot)$.
- Consider a linear space $\mathbb{H}$ of real-valued functions. Furthermore, suppose that $\mathbb{H}$ is a Hilbert space; that is, it is equipped with an inner product operation, and $\mathbb{H}$ is complete with respect to this norm.
- A Hilbert space $\mathbb{H}$ is called reproducing Hilbert space (RKHS) if there exists a kernel function $\kappa : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}$ such that for every $f \in \mathbb{H}$ and for every $\mathbf{x} \in \mathcal{X}$ we can write $f(\mathbf{x}) = \langle f, \kappa(\cdot, \mathbf{x}) \rangle$. For the purpose of our course think that $\mathcal{X}$ is $\mathbb{R}^l$ and $\mathbb{H}$ is $\mathbb{R}^K$.

- There is a unique reproducing kernel Hilbert space (RKHS) associated with every kernel κ (Aronszajn, 1950; Berlinet and Thomas-Agnan, 2004). In this unique association, $\phi(\mathbf{x}) = \kappa(\cdot, \mathbf{x})$ is called the canonical feature map and $\mathbb{H}$ is the feature space.
- Working with reproducing kernel Hilbert spaces we have gained the **kernel trick**, which states that for any two points $\mathbf{x}_i, \mathbf{x}_j$ the inner product of the respective mapping images is written as

$$\langle \phi(\mathbf{x}_i), \phi(\mathbf{x}_j) \rangle = \kappa(\mathbf{x}_i, \mathbf{x}_j). \quad (1)$$

In other words, employing this type of mapping to our problem, we can perform inner product operations in $\mathbb{H}$ in a very efficient way; that is, via a function evaluation performed in the original low-dimensional space!



- The nonlinear task in the original low-dimensional space is mapped to a linear one in the high-dimensional RKHS. Using feature mapping, inner product operations are efficiently performed via kernel evaluations in the original low-dimensional spaces. In practice, the four steps listed above are equivalent to (a) work in the original (low-dimensional Euclidean space) and express all operations in terms of inner products and (b) at the final step substitute the inner products with kernel evaluations.

- The matrix $\mathbf{K} \in \mathbb{R}^{N \times N}$ resulting from the inner products to a dataset, is called the kernel matrix. every kernel matrix $\mathbf{K}$ is symmetric and positive semidefinite!
- The choice of a kernel, as well as the parameters of the kernel, is often chosen using nested cross-validation.
- All is needed is to perform the kernel operations at the final step. Note that the specific form of $\kappa(\cdot, \cdot)$ does not concern the analysis. Once the algorithm for the prediction, $\hat{\mathbf{y}}$, has been derived, one can use different choices for $\kappa(\cdot, \cdot)$. Different choices for $\kappa(\cdot, \cdot)$ correspond to different types of nonlinearity.

Representer Theorem

The representer theorem is of major importance from a practical point of view. It allows us to perform empirical loss function optimization, based on a finite set of training samples, in a very efficient way even if the function to be estimated belongs to a very high (even infinite)-dimensional RKHS $\mathbb{H}$.

Theorem: Let $\Omega : [0, \infty) \rightarrow \mathbb{R}$ be an arbitrary strictly monotonic increasing function. In addition, let $\mathcal{L} : \mathbb{R}^2 \rightarrow \mathbb{R}$ be an arbitrary loss function. Then each minimizer $f \in \mathbb{H}$ of the regularized minimization task

$$\min_{f \in \mathbb{H}} J(f) = \sum_{n=1}^N \mathcal{L}(y_n, f(\mathbf{x}_n)) + \lambda \Omega(\|f\|^2) \quad (2)$$

admits a representation of the form

$$f(\cdot) = \sum_{n=1}^N \theta_n \kappa(\cdot, \mathbf{x}_n), \quad (3)$$

where $\theta_n \in \mathbb{R}$.

- The importance of this theorem is that in order to optimize $J(f)$ with respect to f , one uses the representation $f(\cdot) = \sum_{n=1}^N \theta_n \kappa(\cdot, \mathbf{x}_n)$ and minimization is carried out with respect to the finite set of parameters $\theta_n \in \mathbb{R}$.